

Министерство образования Республики Беларусь
Учреждение образования
Белорусский государственный университет информатики и
Радиоэлектроники

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №2
по курсу “Логические основы интеллектуальных систем”
Вариант 3

Выполнил:
Студент гр. 321701

Струнец Д. П.

Проверил:

Ивашенко В. П.

Минск
2025

Тема:

Решение логических задач на языке Prolog.

Цель:

В соответствии с вариантом необходимо реализовать программу на языке Prolog, решающую поставленную задачу.

Задание:

Два берега реки. На одном из берегов есть три семейные пары, требуется с помощью лодки, вмещающей не более двух человек, переправить всех на другой берег. Нельзя оставлять чужую жену вместе с чужим мужем без присмотра своего супруга.

Дополнительные теоретические сведения:

Prolog – средство написания выполнимых на ЭВМ программ. Язык логического программирования, основанный на логике предикатов первого порядка.

Грамматика языка PROLOG:

<ПРОЛОГ-предложение> ::= <правило> | <факт> | <запрос>

<правило> ::= <заголовок> ‘:-’<тело>

<факт> ::= <заголовок> ‘.’

<запрос> ::= <тело> ‘.’

<тело> ::= <цель> /’,’<цель>/’.’

<заголовок> ::= <предикат>

<цель> ::= <предикат> |<выражение>

<предикат> ::= <имя> / ‘(’<терм> /’,’<терм>/’)’/

<терм> ::= <атом> |<предикат>|<список>

<атом> ::= <переменная> |<число> |<строка>|<имя>

< простой список> ::= ‘[’ <терм> /’,’<терм>/’]’|‘[’]’

<выражение> ::= <терм> /<оператор><терм>/

<список с заголовком> ::= ‘[’ <терм> /’,’<терм>/’]’ < терм>’]

<оператор> ::= ‘is’ | ‘=’ | ‘==’ | ‘\=’ | ‘>=’ | ‘=<’ | ‘=\=’ |

Реализация:

Для решения задачи использовался SWI-Prolog. Для представления состояние нашей задачи будет использоваться функциональный терм `state(list, pos)`, в котором первый параметр определяет список людей на левом берегу второй параметр - местоположение плота. Pos может принимать значения `left` или `right`, что означает нахождение плота у левого или правого берега соответственно. Таким образом, `state([man1, man2, woman1, woman2 ], left)` означает, что плот сейчас находится у левого берега, на левом берегу находится две семейные пары, тогда на правом берегу находятся третья пара. Тогда терминальным состоянием будет `state([], any)`.

В программе используются:

1. Встроенные предикаты:

a) `forall(Cond, Action)`.

Для всех возможных Cond, Action может быть доказано.

b) `member (X, List)`.

Истинно, если X является членом списка List.

c) `subtract(Source, Subtrahend, Result)`.

Вычитает из списка Source содержимое списка Subtrahend и записывает результат в список Result.

d) `sort(Source_list, Destination_list)`.

Истинно, если результат сортировки в обычном порядке элементов списка Source_list может быть унифицирован со списком Destination_list. При этом все повторяющиеся элементы списка Source_list удаляются в полученном после сортировки результате.

e) `format(Format_str, Arguments)`.

Печатает содержимое Format_str, подставляя на указанные места соответствующие элементы списка Arguments.

f) `Action1; Action2`.

Дизъюнкция. Возвращает истину, если истинно Action1 или Action2. Вернёт истину, если истинно и Action1, и Action2.

g) `Action1, Action2`.

Конъюнкция. Возвращает истину, если истинно и Action1, и Action2.

h) `Condition -> Action`.

Импликация. Action выполняется, если истинно Condition.

i) `!`

Отсечение. Применяется, если хватает одного решения, если программист знает о единственности решения, если корректно лишь первое решение и нужно удалить другие решения.

j) =

Достигается вместе с унификацией и может повлечь подстановку.

k) \+

Описывает логическое отрицание.

l) :-

Описывает логическое "если", и служит для разделения двух частей правила: заголовка и тела.

2. Факты:

a) woman(woman1). – факт, утверждающий, что woman1 является женщиной.

b) woman(woman2). – факт, утверждающий, что woman2 является женщиной.

c) woman(woman3). – факт, утверждающий, что woman3 является женщиной.

d) man(man1). – факт, утверждающий, что man1 является мужчиной

e) man(man2). – факт, утверждающий, что man2 является мужчиной.

f) man(man3). – факт, утверждающий, что man3 является мужчиной.

g) family(man1, woman1). – факт, утверждающий, что man1 и woman1 являются семейной парой.

h) family(man2, woman2). – факт, утверждающий, что man2 и woman2 являются семейной парой.

i) family(man3, woman3). – факт, утверждающий, что man3 и woman3 являются семейной парой.

j) all([man1, woman1, man2, woman2, man3, woman3]). – список всех людей.

k) allsafe([_]). – факт, который возвращает истину, если на берегу находится один человек.

l) allsafe([]). – факт, который возвращает истину, если на берегу нет людей.

3. Правила:

a) notsafe_(X, Y) :- woman(Y), man(X), \+ family(X, Y). – истинно, если X и Y не являются семейной парой.

b) notsafe(X, Y) :- notsafe_(X, Y); notsafe_(Y, X). – истинно, если notsafe_(X, Y) или notsafe_(Y, X).

c) safe(X, Y) :- \+ notsafe(X, Y). – истинно, если не notsafe(X, Y).

d) safebridge([X, Y]) :- safe(X, Y), !. – истинно, если safe(X, Y).

e) safebridge([X]) :- man(X); woman(X). – истинно, если man(X) или woman(X).

f) allPairs([H | T], [H, P2]) :- member(P2, T).

Первым и вторым аргументами принимает списки. Во второй список запишется результат, состоящий из двух элементов: H - голова первого списка и P2 - элемент хвоста T первого списка.

g) allPairs([_ | T], P) :- allPairs(T, P).

Первым аргументом и вторым аргументами принимает списки. Во второй список запишет результат allPairs для хвоста первого списка.

В совокупности с предыдущим правилом allPairs позволяет перебрать все возможные пары элементов в каком-либо списке (используются для определения пар, которые поместятся на плоту).

h) allsafe(L) :-

```
forall(
    member(H, L),
    (
        woman(H),
        (
            (
                man(X),
                family(X, H),
                member(X, L)
            );
            \+
            (
                member(M, L),
                man(M)
            )
        );
        man(H)
    )
), !.
```

Истинно, если для всех элементов L выполняется: H – женщина и есть такой мужчина X, что X и H – семейная пара или вовсе нету мужчин, или H – мужчина.

i) step(state(Left1, left), state(Left2, right)) :-

```
step_(state(Left1, left), state(Left2, right)).
```

Определяет следующий шаг с помощью step_ в ситуации, когда плот находится на левом берегу.

j) step(state(Left1, right), state(Left2, left)) :-

```
all(All),
subtract(All, Left1, Right1),
step_(state(Right1, left), state(Right2, right)),
subtract(All, Right2, Left2).
```

Определяет следующий шаг с помощью step_ в ситуации, когда плот находится на правом берегу. После чего определяет кто остался на левом берегу.

k) `step_(state(Left1, left), state(Left2, right)) :-`

```
(
    allPairs(Left1, OnBridge);
    member(A, Left1),
    OnBridge = [A]
),
safebridge(OnBridge),

subtract(Left1, OnBridge, Left2),
allsafe(Left2),

all(All),
subtract(All, Left2, Right),
allsafe(Right).
```

Истинно, если на берегу можно найти такую пару(или одного человека) которая (который) могла (мог) бы переправиться на противоположный берег, при выполнении правил `safebridge`, для пары (человека), переправляющихся (отправляющегося) на плоту, при выполнении правила `allsafe()` для людей на левом и правом берегах после переправы.

l) `notequal(state(L1, P1), state(L2, P2)) :-`

```
\+ (
    P1 = P2,
    sort(L1, L),
    sort(L2, L)
).
```

Истинно, если `state(L1, P1)` не эквивалентен `state(L2, P2)` (если списки состоят из разных элементов или плот находится на разных берегах).

m) `path(Inp, PrevSteps, [step(Inp, S1) | Steps]) :-`

```
step(Inp, S1),
duplCheck(S1, PrevSteps),
path(S1, [step(Inp, S1) | PrevSteps], Steps).
```

Истинно, если существует путь от состояния `Inp`, до терминального состояния `state([], _)`, является целью задачи.

Код программы:

% Лабораторная работа №2 по дисциплине "Логические основы интеллектуальных систем"

% Выполнена студентом группы 221701 БГУИР:

% - Крупский Артём Викторович

% Вариант: 3, последнее изменение: 04.06.2024

% Данный файл является исходным кодом к программе, решающий следующую задачу:

% Два берега реки. На одном из берегов есть три семейные пары, требуется с помощью

% лодки, вмещающей не более двух человек, переправить всех на другой берег.

% Нельзя оставлять чужую жену вместе с чужим мужем без присмотра своего супруга.

% Источники:

% 1. Логические основы интеллектуальных систем. Практикум : учеб.- метод. пособие / В. В. Голенков [и др.].

% – Минск : БГУИР, 2011. – 70 с. : ил. ISBN 978-985-488-487-5.

% 2. SWI Prolog [Электронный ресурс]. -- Режим доступа <https://www.swi-prolog.org/>

% 3. Искусственный интеллект: современный подход, 4-е изд., том 1 Решение проблем:

% знания и рассуждения / Рассел, С. и Питер, Н. – Москва : Диалектика, 2021 – 702 с.

family(man1, woman1).

family(man2, woman2).

family(man3, woman3).

woman(woman1).

woman(woman2).

woman(woman3).

man(man1).

man(man2).

man(man3).

notsafe_(X, Y) :-

woman(Y),

man(X),

\+ family(X, Y).

notsafe(X, Y) :- notsafe_(X, Y); notsafe_(Y, X).

safe(X, Y) :- \+ notsafe(X, Y).

safebridge([X, Y]) :-

safe(X, Y),

!.

safebridge([X]) :-

man(X);

woman(X).

all([

man1, woman1,

man2, woman2,

man3, woman3

]).

allsafe(L) :-

forall(

```

member(H, L),
(
  woman(H),
  (
    (
      man(X),
      family(X, H),
      member(X, L)
    );
    \+
    (
      member(M, L),
      man(M)
    )
  );
  man(H)
)
), !.

```

```

allsafe([_]).
allsafe([]).

```

```

allPairs([H | T], [H, P2]) :-
  member(P2, T).

```

```

allPairs([_ | T], P) :-
  allPairs(T, P).

```

```

step_(state(Left1, left), state(Left2, right)) :-
(
  allPairs(Left1, OnBridge);
  member(A, Left1),
  OnBridge = [A]
),
safebridge(OnBridge),

subtract(Left1, OnBridge, Left2),
allsafe(Left2),

all(All),
subtract(All, Left2, Right),
allsafe(Right).

```

```

step(state(Left1, left), state(Left2, right)) :-
  step_(state(Left1, left), state(Left2, right)).

```

```

step(state(Left1, right), state(Left2, left)) :-
  all(All),
  subtract(All, Left1, Right1),
  step_(state(Right1, left), state(Right2, right)),
  subtract(All, Right2, Left2).

```

```

notequal(state(L1, P1), state(L2, P2)) :-
  \+ (

```



```

P1 = P2,
sort(L1, L),
sort(L2, L)
).
```

```

duplCheck(_, []).
duplCheck(CurrState, [step(State1, _)|T]):-
    notequal(CurrState, State1),
    duplCheck(CurrState, T).
```

```

path(state([], _), _, []).
```

```

path(Inp, PrevSteps, [step(Inp, S1) | Steps]) :-
    step(Inp, S1),
    duplCheck(S1, PrevSteps),
    path(S1, [step(Inp, S1) | PrevSteps], Steps).
```

```

valid_state(state(Left, Bridge)) :-
    (Bridge == left; Bridge == right),
    all(All),
    subtract(All, Left, Right),
    append(Left, Right, AllCurr),
    sort(All, Tmp),
    sort(AllCurr, Tmp).
```

```

printStep(step(state(L1, Pos1), state(L2, _))) :-
    ( Pos1 = left
    -> subtract(L1, L2, Moved),
        all(All),
        subtract(All, L1, R1),
        format('left: ~w | right: ~w~n', [L1, R1]),
        subtract(All, L1, R2),
        format('left: ~w | raft: ~w -> | right: ~w~n~n', [L2, Moved, R2]);
    Pos1 = right
    -> subtract(L2, L1, Moved),
        all(All),
        subtract(All, L1, R1),
        format('left: ~w | right: ~w~n', [L1, R1]),
        subtract(All, L2, R2),
        format('left: ~w | raft: ~w <- | right: ~w~n~n', [L1, Moved, R2])
    ).
```

```

solve(StartLeft, StartBridge):-
    valid_state(state(StartLeft, StartBridge)),
```

```

    sort(StartLeft, SortedStartLeft),
    all(All),
```

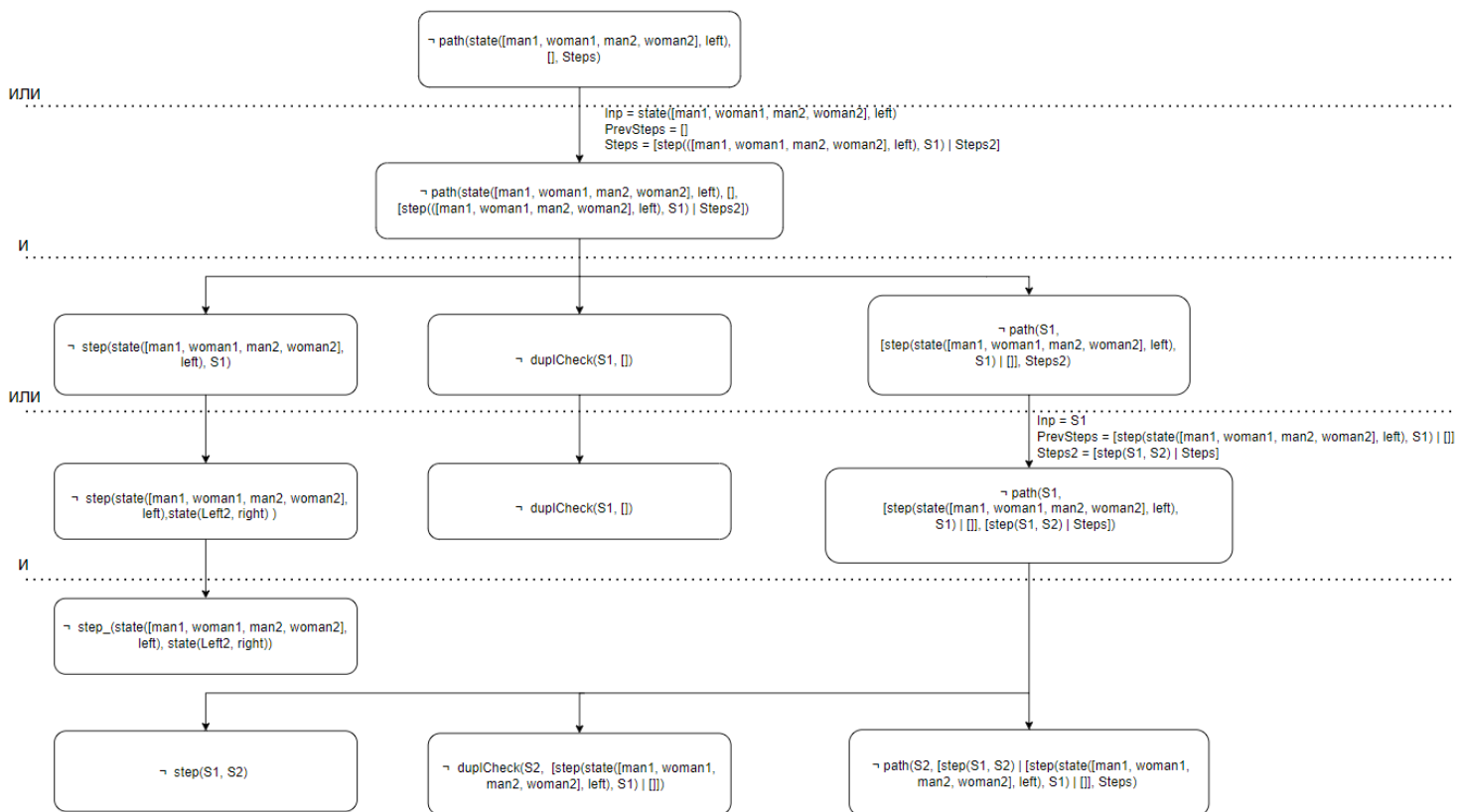
```

    allsafe(SortedStartLeft),
    subtract(All, SortedStartLeft, StartRight),
```

```

    allsafe(StartRight),
    path(state(SortedStartLeft, StartBridge), [], Steps),
    (
        format('~nSolution:~n'),
        forall(member(Step, Steps), printStep(Step))
    ).
```

Дерево логического вывода:



Вывод:

В рамках лабораторной работы была реализована программа решения логической задачи на языке Prolog. Также построено дерево логического вывода, соответствующее данной программе.

Список использованных источников:

1. Логические основы интеллектуальных систем. Практикум: учебно-методическое пособие / В. В. Голенков, В. П. Ивашенко, Д. Г. Колб, К. А. Уваров. – Минск: БГУИР, 2011.
2. SWI-Prolog Documentation [Электронный ресурс].
–<https://www.swi-prolog.org/pldoc/man?section=libpl>.
3. Искусственный интеллект: современный подход, 4-е изд., том 1 Решение проблем: знания и рассуждения / Рассел, С. и Питер, Н. – Москва : Диалектика, 2021 – 702 с.